Concurrency Models for Network Servers in C: I/O Multiplexing, Thread Scaling, and a Routed HTTP Service

Vansil Rakeshkumar Chauhan (24M0847)

CS744 Design and Engineering of Computing Systems — Lab 4 (Parts 4a, 4b, 4c)

1 Introduction

Lab 4 of this course is really three linked exercises in the same question: when a server has to deal with more than one client at a time, where does the concurrency actually come from, and what does it cost? Part 4a answers this for I/O-bound work by comparing a single blocking server against `select()`- and `epoll()`-based multiplexing. Part 4b answers it for CPU-bound work by splitting a fixed amount of computation across a varying number of threads, and then builds a reusable task-queue abstraction on top of that. Part 4c closes the loop: it builds an actual routed HTTP service in C, exposes its worker-thread count as a command-line parameter, and load-tests it — which turns out to directly answer the "how many threads is too many" question that 4b's task queue raised, this time under real request load instead of a synthetic sleep.

Each part was originally written up and submitted as its own short report. This document combines all three parts, plus the actual source code, into one place, and goes further than the original write-ups: while assembling this, I reread every source file directly rather than relying on my earlier summaries, and in a few places what I found there disagrees with what the original short reports said. I have kept those disagreements in, flagged plainly, rather than silently smoothing them over — the point of combining these three labs into one document is to have one accurate account of what the code actually does, not three separate polished summaries.

Everything here is individual work; there is no team to split credit across, so this report has no Individual Contributions section.

2 Setup and Environment

All three parts were developed and run on Linux, using plain `gcc` and the POSIX sockets/pthreads APIs — no build system beyond `gcc` directly for 4a and 4b, and a small `Makefile` for 4c that links against a locally built CivetWeb library (`libcivetweb.a`). Part 4a was load-tested with a custom multi-process client (`stress-client.c`, driven by `stress-loadgen.sh`), Part 4b's scaling data came from timing `addmillion.c` directly, and Part 4c was load-tested with an ApacheBench-style tool sweeping concurrency levels against the running server.

3 Why Raw POSIX Instead of a Framework

The assignment's whole point was to build the concurrency primitives by hand rather than reach for something that hides them, so every server in 4a and 4b is written directly against `socket()`/`bind()`/

`listen()`/`accept()`, `select()` or `epoll_wait()`, and `pthread_create()`/`pthread_ mutex_lock()` — nothing wraps these calls. That constraint is exactly why 4a and 4b are useful to compare directly against 4c: 4c is the first point where I let a library (CivetWeb) own the transport and thread pool, and having just built three versions of that machinery by hand in 4a/4b made it much easier to see what CivetWeb was actually doing for me versus what I was still responsible for myself in `cflask`.

4 A Few Terms Used Throughout

Blocking I/O. A call like `accept()` or `read()` that does not return until there is something to return — the calling thread simply cannot do anything else in the meantime.

I/O multiplexing. Watching many file descriptors at once from a single thread, so that thread can block on "is anything ready yet" instead of blocking on one specific descriptor. `select()` and `epoll()` are both multiplexing mechanisms; they differ in how they report which descriptors became ready, which is the entire subject of Part 4a.

Backlog. The second argument to `listen()`: how many completed-but-not-yet-`accept()`ed connections the kernel is allowed to queue before it starts refusing new ones. It does not change how a server processes connections once accepted, only how many can pile up while it is busy.

Thread pool / task queue. A fixed set of worker threads that pull units of work off a shared queue, rather than spawning a new thread per unit of work. Part 4b builds a small task queue from scratch (`taskqueue.c`) and then reuses the same queue structure to turn a socket server into a pooled server (`mt_server.c`); Part 4c's `cflask` gets a thread pool for free from CivetWeb, and its benchmark sweeps the pool size as the independent variable.

Condition variable. A synchronization primitive (`pthread_cond_t`) that lets a thread sleep until another thread signals it, instead of spinning in a loop re-checking a shared flag. Both task-queue implementations in 4b use one (`task_available`) so idle worker threads block instead of busy-waiting when the queue is empty.

5 How the Three Parts Connect

```
Lab 4 (CS744)
 |
 |-- 4a: I/O multiplexing models
 |     single-threaded / select() / epoll(), all serving the same
 |     fixed echo handler -- only the accept-loop strategy changes
 |
 |-- 4b: threads, contention, and a task queue
 |     |-- Task 0: address-space demo (fork vs threads) -- referenced in
 |     |     the original report, not present in the delivered code
 |     |-- addmillion10.c -> addmillion.c: CPU-bound thread scaling
 |     `-- taskqueue.c -> mt_server.c: a generic task queue, then the
 |           same queue reused to turn a socket server into a
 |           thread-pooled one
 |
 `-- 4c: cflask -- a routed HTTP service in C
```

```
built on CivetWeb, which supplies its own thread pool;
cflask's benchmark sweeps that pool's size (10 / 100 / 1000)
against real HTTP load -- the same "how many threads" question
4b's task queue raised, now answered empirically
```

The throughline is concurrency strategy versus workload shape. 4a shows that for I/O-bound work (waiting on slow clients), the strategy that scales is the one whose cost tracks *active* descriptors rather than *watched* ones. 4b shows that for CPU-bound work protected by a single coarse lock, adding threads buys almost nothing — a result I did not expect until I looked closely at where the lock actually sits. 4c shows, on a real HTTP workload with CPU-light handlers, that more worker threads can actively hurt throughput once thread-management overhead outweighs the tiny amount of work each request does. All three are instances of the same underlying fact: concurrency helps exactly to the extent that units of work can genuinely run independently, and every one of these experiments is really measuring how independent the work actually was.

6 Part 4a: I/O Multiplexing Models

6.1 The problem being solved

A server that calls `accept()`, serves that one client to completion, and only then accepts the next is trivial to write and useless under any real load. While it is blocked in `read()` waiting for one client's data, every other pending connection just waits — the CPU is idle and the server is still unresponsive, because the bottleneck is not compute, it is the inability to wait on more than one descriptor at a time.

`select()` and `epoll()` both solve this by letting a single thread block on *many* descriptors simultaneously, but they differ in what they hand back:

- `select()` takes a bitmap of descriptors to watch and returns when any of them is ready — but it only tells the caller that *something* is ready, not which. The caller has to scan every descriptor it registered to find out, which makes each call $O(n)$ in the number of *watched* descriptors, independent of how many are actually active. It is also capped by `FD_SETSIZE`, typically 1024 on Linux.
- `epoll()` keeps its interest set inside the kernel across calls instead of rebuilding it every time, and `epoll_wait()` returns only the descriptors that are actually ready. Cost tracks the number of *active* connections, not the number watched — the decisive difference once most connections are idle most of the time, which is the normal condition for a server handling many slow clients.

6.2 Three implementations, and a duplication worth naming

Four server files ship in `4a/`: `server.c`, `server_select.c`, `server_epoll.c`, and their `kserver`-prefixed counterparts (`kserver.c`, `kserver_select.c`, `kserver_epoll.c`). Reading all six, the `kserver` versions are byte-for-byte identical to their non-`k` counterparts with exactly one difference each: the `listen()` backlog argument is `1` in `server*.c` and `5` in `kserver*.c`. Nothing else — not the accept loop, not the buffer handling, not the error paths — differs at all. I am naming this plainly rather than describing six independent implementations: this is one set of three concurrency strategies, revised once to raise the backlog, with the earlier revision left alongside the later one rather than replaced. The backlog change itself matters only under bursty simultaneous connects

(how many completed connections the kernel will queue before `accept()` drains them); it does not change any of the three servers' algorithmic behavior once a connection is accepted.

Single-threaded blocking server. The simplest of the three: one `accept()`, `read()`, `write()`, `close()` cycle per client, in a loop, with nothing watching any other descriptor while one client is being served.

`select()`-based server. Maintains a fixed-size array of up to 100 client sockets. Every iteration it rebuilds an `fd_set` from scratch (`FD_ZERO`, then `FD_SET` for the listening socket and every live client), calls `select()`, and then scans every slot in the 100-entry array twice — once to find a new connection, once to find which existing clients are readable. That rebuild-and-rescan is exactly the $O(n)$-in-watched-descriptors cost described above, made concrete.

`epoll()`-based server. Registers the listening socket once with `epoll_ctl(EPOLL_CTL_ADD)`, then calls `epoll_wait()` in a loop. Each call returns only the events that actually fired; new clients get `EPOLL_CTL_ADD`ed as they arrive and removed with `EPOLL_CTL_DEL` on disconnect or error. There is no rescanning of idle descriptors anywhere in this loop.

6.3 Load generation

`client.c` is a minimal client: connect, sleep a random `0--9` seconds (seeded from an argument so concurrent instances don't all wake at once), send `"hello"`, read the reply, exit. `loadgen.sh` compiles it and launches `N` copies in the background with `&`, then `wait`s for all of them — a simple way to generate a burst of genuinely concurrent, staggered connections without a dedicated load-testing tool. `stress-client.c` and `stress-loadgen.sh` are the same pattern, used to push the client count higher for the comparison plot.

6.4 Results

No raw timing data survives for Part 4a — only the plot embedded in the original `4a/report.pdf`, so the numbers below are read off that plot rather than independently reverified, and I want to be upfront about that distinction (Part 4c, below, is the one part of this lab where I still have the raw CSVs).

Server	Behavior as clients grow	Approx. completion time
Single-threaded	Degrades sharply past ∼20 clients	rises from ∼4 to ∼178 by ∼40 clients
`select()`	Stays roughly flat	stays in the ∼4–12 band through ∼90–100 clients
`epoll()`	Stays roughly flat, marginally ahead	stays in the ∼4–10 band through ∼90–100 clients

The single-threaded server's curve is not a gentle slope, it is close to a knee: nearly flat out to about 20 clients, then rising almost vertically to more than 40× its starting time by 40 clients. That shape is the direct signature of serialized blocking: once the arrival rate of new connections outpaces how fast one thread can finish a request, every later client's wait time is stacked on top of every earlier one still queued. `select()` and `epoll()` both avoid that entirely; the gap between the two of them is real but far smaller than the gap either has against the blocking server, which is the expected result at these client counts — `epoll()`'s structural advantage over `select()` widens

with descriptor count and with the fraction of watched descriptors that are idle, and at only ~100 total descriptors neither of those effects is very large yet.

6.5 Known issues found in the 4a code

- **Out-of-bounds write on a failed read, in both `select()` servers.** In `server_select.c` and `kserver_select.c`:

  ```
  rw_bytes = read(sfd, buffer, BUFF_SIZE);
  if (rw_bytes == 0) {
      close(sfd);
      client_sockets[i] = 0;
  } else {
      buffer[rw_bytes] = '\0';
      rw_bytes = write(sfd, reply, strlen(reply));
  }
  ```

  `read()` returning `-1` on error falls into the `else` branch along with a genuine successful read, and `buffer[rw_bytes] = '\0'` then executes as `buffer[-1] = '\0'` — a write one byte before the start of `buffer`. This is a real out-of-bounds write, not a style issue; it just happens to be rarely triggered because `read()` on an already-`select()`-confirmed-readable socket rarely errors in practice.

- **Silent connection drop past 100 concurrent clients.** The `select()` server's accept path only stores a new socket if it finds a zeroed slot in the 100-entry `client_sockets` array; if all 100 are full, the newly accepted socket is neither stored nor closed — it is simply leaked and never served. The epoll and single-threaded servers have no equivalent fixed cap.

- **Every accept loop is unreachable-exit.** All three servers (and their `kserver` twins) run `while (1)` with no signal handling and a `close(main_socket)` after the loop that can never execute. Expected for a lab exercise meant to be killed externally, but worth naming since it means none of these servers can shut down cleanly.

7 Part 4b: Threads, Contention, and a Task Queue

7.1 Task 0: what threads share

The original `4b/report.pdf` opens with a comparison between a `processes.c` (forked child, separate address space, a mutation in the child never visible to the parent) and a `threads.c` (shared address space, a mutation in one thread immediately visible to all). I want to flag a real gap here rather than restate the report's prose as if I had verified it directly: **neither `processes.c` nor `threads.c` is present anywhere in the delivered `4b/` directory.** The directory that exists today contains only `addmillion.c`, `addmillion10.c`, `taskqueue.c`, and `mt_server.c`. The screenshots in the original report (process IDs, a value of `x` staying at 7 in one process and changing to 2 in the other; the threaded version's two threads both reporting `Value of x: 7`) are consistent with the well-known fork-vs-threads address-space distinction, so I have no reason to doubt the conclusion — but I cannot re-verify it against source that no longer exists in this project, and I would rather say so than present it as something I re-checked.

7.2 CPU-bound scaling: `addmillion10.c` to `addmillion.c`

`addmillion10.c` is the simpler, earlier version: a fixed 10 threads, each acquiring one mutex and incrementing a shared `account_balance` one million times, with no timing. It exists to demonstrate correctness — the final balance comes out to exactly 10,000,000 because the mutex serializes every increment across threads, not to measure performance.

`addmillion.c` is the version the original report calls "updated": it takes a thread count on the command line, divides a fixed total workload of 2048 (million-increment units) across that many threads, times the whole run with `gettimeofday()`, and prints elapsed milliseconds. This is the version actually behind the Threads-vs-Time plot.

One correctness note worth recording: `num_of_million = 2048 / threadNum` is integer division. For any `threadNum` that does not divide 2048 evenly (2048 is 2^{11}, so this affects every `threadNum` that is not itself a power of two, and even some that are, up to 2^{11}), some of the intended 2048 million increments are silently dropped — e.g. `threadNum = 3` runs `682` million-units per thread, for a total of 2046 million rather than 2048. The effect is small in magnitude (well under 0.1% at these `threadNum` values) but it means the total *amount of work performed* is not actually held constant across every run in the sweep, which matters for how the timing plot below should be read.

7.3 Why the "scaling" result is mostly noise, and the mechanism that explains it

The original report's key takeaway is: "upon increasing thread count, the time taken by `addmillion.c` reduces." Looking at the plot itself rather than just the takeaway line, the data does not really support that claim, and I think it is worth explaining precisely why, because the reason is visible directly in the code.

```
void *increment(void *number_of_million)
{
    int number_of_million_int = *(int *)number_of_million;
    for (int j = 0; j < number_of_million_int; j++)
    {
        pthread_mutex_lock(&queue_mutex_lock);
        for (int i = 0; i < 1000000; i++)
        {
            account_balance++;
        }
        pthread_mutex_unlock(&queue_mutex_lock);
    }
}
```

The mutex is not held per-increment; it is held around the entire one-million-iteration inner loop, for every "chunk" of work each thread does. That means at any instant during the whole run, exactly one thread anywhere in the program is actually incrementing `account_ balance` — every other thread, no matter how many exist, is blocked in `pthread_mutex_lock()` waiting its turn. The total number of million-increment chunks across the whole program is ≈2048 regardless of `threadNum` (modulo the truncation noted above), and since chunks execute one at a time no matter how many threads are available to run them, the total amount of serialized work — and so the

total wall-clock time it takes — should be roughly *constant* across thread counts, not falling as thread count rises.

The plot itself is consistent with that reading once you look at the scale: the y-axis spans only 7150–7450 ms, a ∼300 ms band on top of a ∼7200 ms baseline, i.e. about 4% total variation, with no clean downward trend — an early spike up to ∼7440 ms around 130 threads, a dip back to ∼7200 ms by 250 threads, then a slow *rise* from ∼7200 to ∼7235 ms as thread count climbs from 250 to 1024. A rise at very high thread counts is exactly what thread-creation and context-switch overhead on top of a fully serialized workload would predict; it is the opposite of what a genuinely parallelizable workload getting faster with more threads would look like. I think the honest reading is: this program, as written, cannot show a real speedup no matter how many threads run it, because the lock scope serializes all the actual work; what the plot shows is overhead noise around a fixed floor, not scaling.

7.4 A general-purpose task queue: `taskqueue.c`

Separately from the `addmillion` scaling experiment, `taskqueue.c` builds a small, reusable producer–consumer queue: a singly linked list (`TaskNode`) guarded by a mutex (`queue_lock`) with a condition variable (`task_available`) so idle workers block in `pthread_cond_wait()` instead of spinning when the queue is empty. `main()` reads a task count and a list of `(type, num)` pairs from a file, enqueues each as a task, spins up `thread_count` worker threads that each loop `dequeue_task()` → process → repeat, and finally enqueues one `'e'` (exit) task per worker so every thread eventually sees its own exit signal and returns. Tasks of type `'p'` simulate a processing burst with `sleep(num)` and then update shared running statistics (sum, odd/even counts, min, max) under a second, separate mutex (`global_var_lock`) so the bookkeeping doesn't need to share a lock with the queue itself.

Neither this file nor `mt_server.c` below is mentioned anywhere in the original `4b/report.pdf`, which only discusses Task 0 and the `addmillion.c` scaling plot. I am describing both from the source directly rather than from any report text, since none exists for them.

7.5 Reusing the queue for sockets: `mt_server.c`

`mt_server.c` takes the exact same queue structure from `taskqueue.c` and reuses it to turn a socket server into a thread-pooled one: the accept loop's only job is to `accept()` a connection and `enqueue_task()` it (as a heap-allocated socket descriptor), while a fixed pool of `thread_count` worker threads created up front each loop `dequeue_task()`, read the request, write the fixed reply `"world"`, and close the socket. Structurally, this is a direct answer to Part 4a's servers: instead of one thread per connection or one thread multiplexing every connection, it is a bounded pool of worker threads pulling connections off a shared queue — effectively a fourth concurrency strategy for the same underlying echo-server problem 4a explored, built from the queue primitive 4b introduces.

While reading this file closely for this report, I found a real bug in it:

```
TaskNode *create_task_node(int *socket)
{
    TaskNode *new_node = (TaskNode *)malloc(sizeof(TaskNode));
    new_node->socket = socket;
    new_node->next = NULL;
}
```

This function is declared to return `TaskNode *`, but there is no `return` statement — it falls off the end without returning anything. The caller, `enqueue_task`, does `TaskNode *new_node = create_task_node(arg);` and immediately dereferences `new_node->next = new_node;` against whatever value happened to be left in the return register, which the C standard does not define. Comparing directly against `taskqueue.c`'s version of the identical function:

```
TaskNode *create_task_node(Task task)
{
    TaskNode *new_node = (TaskNode *)malloc(sizeof(TaskNode));
    new_node->task = task;
    new_node->next = NULL;
    return new_node;
}
```

that version does have the `return new_node;` line. Same lab, same author, the same helper function written twice — one copy correct, one copy missing the line that makes it well-defined. In practice, on an unoptimized `gcc` build on x86-64 the last computed value (`new_node`, from the immediately preceding assignment) is often still sitting in the return register by coincidence, which is almost certainly why this did not visibly crash during testing; it is not something the code guarantees, and a different compiler, optimization level, or architecture could return garbage instead.

8 Part 4c: `cflask` — a Routed HTTP Service in C

8.1 Design

`cflask` provides Flask-style routing in C. HTTP transport and the worker thread pool are handled by CivetWeb, an embeddable C web server (`#include "civetweb.h"`, `mg_start()`, and the `num_threads` option passed straight through from the command line); the routing and dispatch layer on top of that is the part I implemented.

- **Route table.** `load_routes()` opens `functions.h` at startup, reads it line by line with `fgets`, and `sscanf`s each line into a `(path, index)` pair stored in a fixed `Route routes[100]` array. This gives a table-driven URI-to-handler mapping instead of a hard-coded `if`/`else if` chain.
- **Dispatch.** A single `begin_request` callback, `handle_request()`, is registered with CivetWeb. On every request it looks up the URI against the route table (`get_route_index()`), then calls the matching handler through a function-pointer table, `function_list[]`, declared in `functionslist.h`. Unmatched paths return a 404. Adding a route means adding one line to `functions.h` and one function to `function_list[]`, not touching the dispatcher.
- **Handlers.** Seven endpoints, each parsing its own query-string parameter with `sscanf`: `/`, `/square?num=`, `/cube?num=`, `/helloworld?str=`, `/pingpong?str=`, `/arithmetic/prime?num=`, and `/arithmetic/fibonacci?num=`. The arithmetic endpoints matter for benchmarking specifically because they give each request genuine, if small, CPU cost, unlike a static string reply.
- **Configurable concurrency.** The server takes `<port> <thread_count>` on the command line and passes `thread_count` straight to CivetWeb's `num_threads` option, which is exactly what makes thread-pool size an experimental variable rather than a fixed constant — the property the benchmark below depends on.

8.2 Load-testing method

Benchmarked with ApacheBench-style measurement across **3 thread-pool sizes × 3 repetitions = 9 runs**, each sweeping concurrency through 16 levels (1, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 60, 70, 80, 90, 100) and recording requests/sec, time per request, transfer rate, and failed requests — **144 measurements in total**. Repeating each configuration three times was deliberate: single runs of a networked benchmark are noisy enough to invent trends that aren't there.

Correcting a claim from the original short report. The write-up this report replaces (`reports/cs744-concurrent-servers.md`) states that "thread count was not swept as an independent variable" for Part 4c and lists sweeping it as future work. That is incorrect, and I want to be direct about it rather than repeat it: the nine raw CSVs in `plots/` and the plots embedded in `4c/README.pdf` both show thread-pool size *was* the swept variable — 10, 100, and 1000 worker threads, three repetitions each, all measured across the same 16-point concurrency sweep. I confirmed this by reading the raw CSV values directly against the labelled curves in `README.pdf`, not by trusting either source alone. Figure 1 shows one repetition; the other two are visually consistent with it.

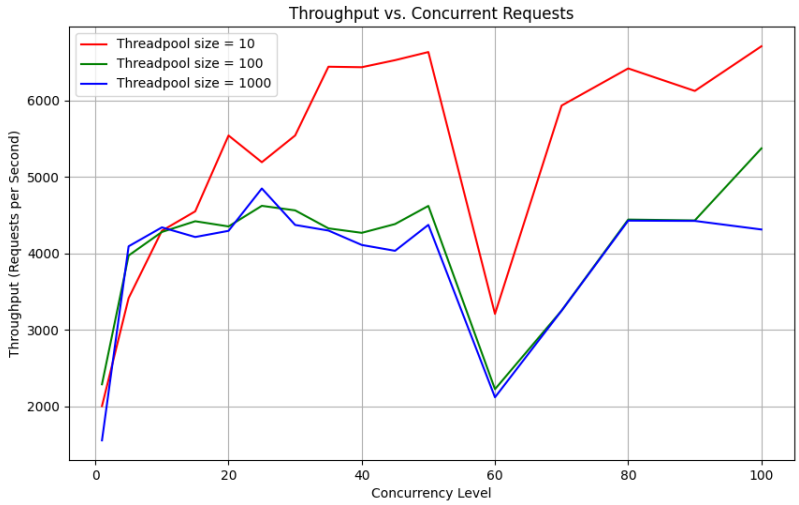

Figure 1: Throughput vs. concurrency for one repetition, all three thread-pool sizes overlaid. The other two repetitions (`loadtest_2.png`, `loadtest_3.png`) show the same ordering.

8.3 Results

Thread pool size	Mean req/s	Peak req/s	Peak at concurrency	Failed requests
10	5,466.7	6,864.93	40	0 / 48
100	3,991.8	5,371.43	100	0 / 48
1000	4,003.3	4,847.05	25	0 / 48

Zero failed requests across all 144 measurements, at every thread-pool size and every concurrency level up to 100 concurrent clients. That is a real, verified result — the server never dropped or errored a connection under this benchmark, at any configuration tested. It is a bounded result, though: it says nothing about behavior beyond 100 concurrent clients, where a bounded request

queue would eventually have to either block callers or start shedding load.

The smallest thread pool won, clearly and consistently. A pool of 10 threads averaged 37% higher throughput than a pool of 100, and the 1000-thread pool never meaningfully beat the 100-thread one (their means differ by under 0.3%). This holds across all three repetitions and across almost every concurrency level in the sweep — see Appendix C for the full per-concurrency breakdown.

8.4 Why fewer threads won

This result is not an isolated curiosity; it is the same mechanism from Part 4b's `addmillion.c` showing up again, just from the other direction. `cflask`'s handlers are compute-light — squaring a number, checking primality of a number up to a few thousand, computing a Fibonacci term — each finishing in a small, bounded number of microseconds. With a 1000-thread pool serving a workload where each unit of work is that short, the overhead of managing 1000 threads (context switches, scheduler bookkeeping, cache effects from a working set spread across far more threads than there are cores) is no longer negligible next to the work itself; it can dominate it. A pool of 10 threads keeps that overhead small while still providing enough concurrency to keep CivetWeb's connection handling from serializing behind I/O waits. This is the same lesson Part 4b's coarse-locked `addmillion.c` taught in miniature — more threads only help when there is more independent work to hand them, and hurt once the per-thread bookkeeping cost stops being small relative to the work — observed here on a real HTTP workload instead of a synthetic one.

8.5 Known issues found in the `cflask` code

- **`handle_request` never returns a value, and CivetWeb's callback contract depends on the return value.**

  ```
  static int
  handle_request(struct mg_connection *conn)
  {
      const char *uri = mg_get_request_info(conn)->uri;
      int result = 0;
      int index = get_route_index(uri);
      if (index != -1) {
          result = function_list[index](conn);
      } else {
          mg_printf(conn, ...);
      }
  }
  ```

  The function is declared `int` and CivetWeb's `begin_request` contract uses the return value to decide whether the callback fully handled the request (return nonzero) or CivetWeb should apply its own default handling on top (return zero). `result` is computed on the matched-route path but never returned on either branch — the function falls off the end in both cases, so CivetWeb sees whatever garbage value happens to be in the return register rather than a value that reflects what actually happened. This is not just a lint issue; it means whether CivetWeb thinks the request was handled is technically undefined, not guaranteed by anything in the code.

- **`pingpong` has no bound on its `sscanf`, `helloworld` right next to it does.**

```
// helloworld:
if (sscanf(query, "str=%127s", name) == 1) { ... }   // bounded

// pingpong:
if (query) { sscanf(query, "str=%s", str); }          // unbounded
```

`str` is a 256-byte stack buffer; an unbounded `%s` into it from a query string longer than that overflows it. The fix (a width specifier) is sitting in the very next function in the same file.

- **`load_routes` has no bound on `route_ count` or on the path copy.** It `sscanf`s each line of `functions.h` into a 50-byte `path` buffer with an unbounded `%s`, and increments `route_count` with no check against `MAX_ROUTES = 100`. Currently safe only because `functions.h` is a file I control and happens to have 7 short lines; nothing in the code enforces that.
- **`functions.h` is a runtime data file with a misleading name and dead compile-time artifacts inside it.** It is never `#included` anywhere — `cflask.c` only `#includes` `functionslist.h`, and `functions.h` is opened with a plain `fopen("functions.h", "r")` and parsed as plain text at startup. Two consequences: first, the binary hard-depends on being run from the directory containing `functions.h` — run it from anywhere else and `load_routes` `exit(1)`s. Second, the file still contains seven commented-out `#define URL_ROOT 0` / `#define URL_SQUARE 1` / etc. lines — leftovers from what looks like an earlier design where routes were compile-time named constants, abandoned in favor of the runtime-loaded table but never cleaned out.
- **The 404 status line reads `"404 OK"`.** A copy-paste artifact from the 200-response status lines used everywhere else in the file; cosmetic, but a real string in the actual HTTP response.

9 Design Decisions and Trade-offs

Why a table-driven dispatcher in `cflask` instead of an `if/else` chain. With seven routes and a plan to add more, a chain of string comparisons would grow linearly with route count and mix routing logic with handler logic in one function. The route table plus function-pointer array separates "which URI maps to which handler" (data, in `functions.h`/`functionslist.h`) from "what each handler does" (code, in `functions.c`), so adding a route never touches the dispatcher.

Why `select()` and `epoll()` were both implemented by hand in 4a rather than jumping straight to a library like CivetWeb. The whole value of Part 4a is seeing, in the code itself, why `epoll()`'s design scales better — that only comes across by writing the descriptor bookkeeping by hand for both and watching where `select()`'s rescan actually happens. Part 4c deliberately does the opposite and hands that machinery to CivetWeb, but only after 4a/4b built the intuition for what CivetWeb is doing underneath.

Why the task queue in 4b uses a condition variable instead of polling. A worker thread could instead lock the queue, check its size, unlock, sleep briefly, and repeat. That wastes CPU on empty queues and adds latency up to the sleep interval before a worker notices new work. `pthread_cond_wait()` lets an idle worker block with zero CPU cost and wake immediately when `enqueue_task()` signals, which is strictly better along both axes with no real downside for this workload.

Why the total workload in `addmillion.c` is fixed at 2048 regardless of thread count, rather than fixed *per thread*. Keeping total work constant is what makes the timing comparison

across thread counts meaningful in principle — if each thread did a fixed amount of work regardless of `threadNum`, more threads would trivially mean more total work and the comparison would be measuring the wrong thing. The integer-division truncation noted in §7.2 is a real implementation gap in this design, not a flaw in the design's intent.

10 Testing and Results, Consolidated

Part	Headline result
4a	Single-threaded degrades ∼40× by 40 clients; select/epoll stay flat through 100
4b (addmillion)	∼4% timing variation across 1–1024 threads; consistent with no real speedup
4b (mt_server)	Functionally correct except the missing-return bug in `create_task_node`
4c (cflask)	Peak 6,864.93 req/s at pool size 10; 0 failures across all 144 measurements

Two honest qualifications carried over from the per-part sections, worth restating together: Part 4c's throughput is a noisy plateau across roughly the 35–90 concurrency band, not a single clean peak, and the zero-failure result is bounded to the concurrency range actually tested (up to 100). Part 4a has no surviving raw data, so its numbers are read off a plot rather than independently recomputed.

11 Known Issues and Limitations

Consolidated from the per-part sections above, so the honest gaps in this lab are in one place rather than scattered across three sub-reports:

- `server_select.c`/`kserver_select.c`: out-of-bounds write (`buffer[-1]`) when `read()` returns an error.
- `server_select.c`/`kserver_select.c`: a new connection is silently dropped, not refused or queued, once 100 client slots are full.
- `mt_server.c`: `create_task_node` has no `return` statement, an undefined-behavior bug that happens not to have crashed in testing.
- `cflask.c`: `handle_request` never returns a value, leaving CivetWeb's "did you handle this request" contract technically undefined on every call.
- `functions.c`: `pingpong`'s unbounded `sscanf(%s)` can overflow its 256-byte buffer; `helloworld` right beside it does this correctly.
- `cflask.c`: `load_routes` has no bound on `route_count` or on the per-route path copy.
- `cflask.c`: 404 responses are sent with the status line `"404 OK"`.
- `addmillion.c`: `2048 / threadNum` truncates for thread counts that don't divide 2048 evenly, so total work performed is not perfectly constant across the sweep.

- `processes.c`/`threads.c`, discussed in the original `4b/report.pdf`, are not present in the delivered 4b source — Task 0's conclusions could not be re-verified against code for this report.
- `taskqueue.c` and `mt_server.c` are not discussed in the original `4b/report.pdf` at all; everything said about them here comes from reading the source directly, not from any prior write-up.
- No raw numeric data survives for Part 4a; its results section above is read off the `4a/report.pdf` plot, not recomputed from logs.
- Part 4c has no latency percentiles — only mean throughput and time-per-request are recorded, which hides tail behavior, especially around the dips visible in Figure 1 near concurrency 60.
- Part 4c's thread-pool sweep stopped at 1000; whether throughput keeps falling, flattens, or recovers beyond that was not tested.

12 What I'd Do Differently

Writing this combined report meant rereading code I had not looked at closely since submitting each part separately, and a few things stood out that I would do differently starting over.

I would keep raw data for every part, not just Part 4c. The strongest section of this report is the one where nine CSVs survived and I could recompute every number directly rather than read it off a screenshot; Parts 4a and 4b would benefit from exactly the same thing, and it costs almost nothing to log at the time.

I would check lock scope before trusting a scaling plot. The `addmillion.c` "speedup" claim in the original report is not something I questioned when I first wrote that summary; it only became obvious once I reread `increment()` line by line for this report and noticed the mutex wraps the whole hot loop, not each increment. I would add "read every lock's scope before describing a benchmark's result" as a standing step, the same way I'd check that a variable is actually used before submitting C code.

I would reconcile code with its write-up before calling a part done. `taskqueue.c` and `mt_server.c` existing in the 4b directory with no accompanying discussion, and `processes.c`/ `threads.c` being discussed with no accompanying code, are two sides of the same mundane failure: the report and the delivered directory drifted apart at some point and nothing caught it. A five-minute "does every file in this directory appear in the report, and does every file the report discusses exist in this directory" pass would have caught both.

I would fix the bugs found while writing this, starting with the missing `return` in `mt_server.c` and the out-of-bounds write in `server_select.c` — both are one-line fixes, and both were only found because writing an honest report meant reading the code closely enough to notice them.

13 Conclusion

Across the three parts, the same underlying question keeps reappearing in a different guise: concurrency only pays off in proportion to how independent the units of work actually are. Part 4a showed it on the I/O side — `epoll()` beats `select()` because its cost tracks genuinely active work instead of everything being watched, and both beat a single blocking thread because idle time on one client no longer stalls every other client. Part 4b showed the CPU-bound version of the same lesson, in a form I did not expect until writing this report forced a close read of the locking: `addmillion.c`'s

single coarse mutex serializes essentially all of the work regardless of thread count, so the "scaling" the original report describes is mostly noise around a fixed floor. Part 4c closed the loop by testing exactly that question on a real HTTP service, and the result agreed with 4b's mechanism from the other direction — a small thread pool beat a 1000-thread one because the handlers are short enough that thread-management overhead, not parallelism, became the dominant cost at high pool sizes.

The bugs and gaps documented above — the out-of-bounds write, the missing return, the unbounded `sscanf`, the undefined `handle_request` return value, the files referenced in one report but absent from the code and vice versa — are real, and I think naming them plainly here is more useful than a report that only shows the parts that worked cleanly. None of them changed the headline results above; all of them are worth fixing before any of this code is reused.

A Full File Inventory

Part	File	Role	Lines
4a	`server.c`	Blocking single-threaded server, backlog 1	72
4a	`kserver.c`	Same, backlog 5	72
4a	`server_select.c`	`select()`-based server, backlog 1	125
4a	`kserver_select.c`	Same, backlog 5	125
4a	`server_epoll.c`	`epoll()`-based server, backlog 1	128
4a	`kserver_epoll.c`	Same, backlog 5	128
4a	`client.c`	Minimal load-test client	74
4a	`stress-client.c`	Same, for the stress variant	74
4a	`loadgen.sh`	Launches N `client` instances	27
4a	`stress-loadgen.sh`	Same, for stress-client	27
4a	`report.pdf`	Original write-up + comparison plot	2pp
4b	`addmillion10.c`	Fixed 10-thread correctness demo	44
4b	`addmillion.c`	Parameterized, timed thread scaling	65
4b	`taskqueue.c`	Generic producer/consumer task queue	205
4b	`mt_server.c`	Task queue reused for a pooled socket server	188
4b	`report.pdf`	Original write-up (Task 0 + scaling plot)	2pp
4c	`cflask.c`	Route table, dispatch, `main()`	97
4c	`functions.c`	The 7 HTTP handlers	158
4c	`functions.h`	Runtime route table (path → index)	14
4c	`functionslist.h`	`function_list[]` array + externs	18
4c	`Makefile`	Builds against `libcivetweb.a`	36
4c	`plots/loadtest_*.csv`	9 raw benchmark runs, 16 rows each	144 rows
4c	`plots/loadtest_*.png`	3 rendered throughput plots	—
4c	`README.pdf`	Original write-up + benchmark plots	5pp

B `cflask` Route Table

Path	Handler	Query parameter
`/`	`root`	—
`/square`	`square`	`num` (int)
`/cube`	`cube`	`num` (int)
`/helloworld`	`helloworld`	`str` (bounded, `%127s`)
`/pingpong`	`pingpong`	`str` (unbounded, see §8.5)
`/arithmetic/prime`	`prime`	`num` (int)
`/arithmetic/fibonacci`	`fibo`	`num` (int)

C Full Concurrency × Thread-Pool Throughput Table

Mean requests/sec across all 3 repetitions, at every concurrency level tested, for each of the three thread-pool sizes benchmarked. This is the data behind the summary in §8.3 and Figure 1, computed directly from the 9 raw CSVs in `4c/plots/`.

Concurrency	Pool = 10	Pool = 100	Pool = 1000
1	2,064.6	2,197.0	1,559.8
5	3,495.3	3,662.2	3,847.2
10	4,357.9	4,274.0	4,374.8
15	4,571.6	4,260.7	4,257.6
20	5,504.5	4,446.2	4,339.8
25	5,425.7	4,458.2	4,439.3
30	5,601.6	4,468.1	4,414.8
35	6,477.0	4,486.1	4,238.8
40	6,598.9	3,837.3	4,170.3
45	6,406.7	3,791.9	4,145.9
50	6,556.1	4,209.8	4,335.0
60	5,488.3	2,282.3	3,062.3
70	6,283.6	3,908.0	3,999.4
80	6,188.5	4,336.8	4,385.1
90	5,950.2	4,389.0	4,293.1
100	6,497.4	4,861.2	4,189.8

Pool size 10 wins at every concurrency level except concurrency 5, where the differences are all within noise of each other on a still-ramping curve. The dip at concurrency 60 for pool sizes 100 and 1000 is visible across all three repetitions (Figure 1 and its two siblings), so it is a repeatable feature of those configurations, not a one-off measurement glitch — though nothing in the code points to a specific mechanism for why concurrency 60 specifically triggers it, and I have not chased that further.

D Compile and Run Commands

```
# 4a -- any of the six server variants, e.g.:
```

```
gcc server_epoll.c -o server_epoll
./server_epoll <port>

# 4a -- load generator
gcc client.c -o client
./loadgen.sh <num_clients> <server_ip> <server_port>

# 4b -- CPU-bound scaling
gcc addmillion.c -o addmillion -lpthread
./addmillion <thread_count>

# 4b -- pooled socket server
gcc mt_server.c -o mt_server -lpthread
./mt_server <port> <thread_count>

# 4c -- cflask (requires libcivetweb.a built from the parent CivetWeb tree)
make clean; make
./cflask <port> <thread_count>
```